String Indexing (Exercises)

1. `'python'[0]` `'p'`
2. `'python'[1]` _____
3. `'python'[3]` _____
4. `'python'[-1]` _____
5. `'python'[-3]` _____

String Indexing

Strings of text may be surrounded by single quotation marks (`'`).

Putting a number in square brackets (`[]`) after a string gives the single character at that *index* (position). Indices are zero based, so the first character is at index 0.

Negative indices start from the end of the string, so index -1 is the last character, index -2 is the character before that, and so on.

String Slicing (Exercises)

1. `'abandon'[0:2]` `'ab'`
2. `'abandon'[1:5]` _____
3. `'infinite'[2:100]` _____
4. `'incandescent'[3:8:2]` _____
5. `'airspace'[:3]` _____
6. `'airspace'[3:]` _____
7. `'remote'[4:0:-1]` _____
8. `'duplicate'[::]` _____
9. `'straw'[::-1]` _____

String Slicing

Up to three numbers separated by colons (`:`) within square brackets after a string give a *slice* of the string (a shorter string):

- The first number gives the *start* index of the slice.
- The second number gives the *stop* index. This is exclusive, so the slice includes character up to but not including the stop index.
- The third number gives the *step*. The index is advanced by this much for each character of the slice, so (for example) a step of 2 will include every other character. If the step is negative, the slice works backward from the start to (but not including) the stop.

Any or all of the three numbers specifying a slice may be omitted.

- If no step is provided, the slice starts at the beginning of the string.
    - ... unless the step is negative, in which case the slice starts at the end of the string.
- If no stop is provided, the slice stops at the end of the string.
    - ... unless the step is negative, in which case the slice stops at the beginning of the string.
- If no step is provided, a step of 1 is used. The second colon can be omitted in this case.

If the stop is beyond the end of the string, the slice ends where the string ends.

Arithmetic Operators (Exercises)

1. `2 + 2` 4
2. `2 + 3` _____
3. `4 - 1` _____
4. `3 - 8` _____
5. `2 * 4` _____
6. `8 / 2` _____
7. `7 / 2` _____
8. `7 // 2` _____
9. `34 // 10` _____
10. `34 % 10` _____
11. `2 ** 3` _____
12. `10 ** 20` _____
13. `2 + 3 * 5` _____
14. `2 + (3 * 5)` _____
15. `(2 + 3) * 5` _____
16. `1 + 2 * 3 - 4 ** 2 / 4` _____

Arithmetic Operators

Numbers can be combined with familiar arithmetic *operators*. `+` and `-` perform addition and subtraction. The multiplication operator is `*`.

There are three division operators:

- `/` performs standard division. The answer always includes a decimal point, even if it happens to be an integer.
- `//` performs integer division, throwing away any remainder.
- `%`, the *modulo* or *remainder* operator, keeps *only* the remainder.

The `**` operator performs exponentiation, so `5 ** 2` computes 5^2.

Some operators have higher precedence than others, so they are performed first:

- `**` has the highest precedence
- `*`, `/`, `//`, and `%` have the next highest
- `+` and `-` have the lowest

Parentheses can be used to ensure that particular operations are performed earlier.

Variables (Exercises)

1. `x = 3`

   `x` 3

2. `x + 2` _____

3. `y = 4`

   `y` _____

4. `x + y` _____

5. `color = 'blue'`

   `color` _____

6. `score = 5`

   `score = 10`

   `score` _____

7. `a = 1`

   `b = a`

   `a = 2`

   `b` _____

8. `n = 10`

   `n = n + 1`

   `n` _____

Variables

A *variable* is a name for something. It can be *assigned* a value with the = operator. For example, `x = 3` makes `x` a name for the number 3.

Once it has been assigned a value, the variable can be used in place of the value.

A variable can be redefined by assigning it a new value.

Note that = is not a declaration that two things are the same. It is an assignment that makes the variable name on the left a name for the value on the right.

Assignment makes the variable stand for the *value* on the right side, not the code that produced that value. Specifically:

- If a variable is defined in terms of another variable, later changes to the other variable don't affect the first one.
- A variable can be defined in terms of its current value. For example, `x = x + 1` gets the current value of `x`, adds 1, and makes `x` a name for that new value.

String Operators (Exercises)

1. `'hot' + 'dog'` `'hotdog'`
2. `'shoe' + 'string'` _____
3. `'ma' * 2` _____
4. `'co' * 2 + 'a'` _____
5. `'i' in 'team'` _____
6. `'on' in ('no' * 2)` _____
7.
```
a = 'brown'
b = 'toad'
a[1:3] + b[2:]
```
_____

String Operators

Two strings can be concatenated with `+`.

Multiplying a string by a positive integer (using `*`) concatenates multiple copies of it.

If `a` and `b` are strings, `a in b` is true if the string `a` appears somewhere within `b`.

Quotation Marks (Exercises)

1. `"hello"` `'hello'`
2. `'goodbye'` _____
3. `"don't"` _____
4. `'Eddie "The Eagle" Edwards'` _____
5. `'You "can\'t" avoid it.'` _____
6. `"... or \"can\" you?"` _____
7. `'Maybe \'sometimes\' you can.'` _____

Quotation Marks

Strings may be delimited by either single (`'`) or double (`"`) quotation marks.

Single quotation marks can be used in a string delimited by double quotation marks, and vice versa.

If it is necessary to use the kind of quotation mark that delimits a string inside that string, precede it with a backslash (`\`).

When it displays strings, Python prefers single quotation marks, but will use double quotation marks to avoid a backslash.

Calling Functions (Exercises)

1. `abs(-3)` 3
2. `abs(12)` _____
3. `len('python')` _____
4. `max(2, 3)` _____
5. `min(2, 3)` _____
6. `round(3.14159)` _____
7.
```
a = 'snake'
b = 'language'
max(len(a), len(b))
```
_____
8.
```
s = 'python'
s[0:len(s)]
```
_____
9. `max(1, min(2, 3))` _____
10. `sorted(s)` _____
11. `s` _____

Calling Functions

To *call* a *function*, use the name of the function followed by round parentheses (`()`). If the function takes any *arguments*, put them between the parentheses, separated by commas. The value returned by the function call depends on the function and the arguments.

Here are some of Python's built-in functions:

- `abs(x)` returns the absolute value of the number `x`.
- `len(s)` returns the length of the string `s`.
- `max(x, y)` returns the larger of `x` and `y`.
- `min(x, y)` returns the smaller of `x` and `y`.
- `round(x)` returns the number `x` rounded to the nearest integer. If `x` is exactly halfway between two integers, `round` returns the even one.
- `sorted(s)` returns a sorted version of `s`. It does not modify `s` itself, but returns a new string.

Many of these functions can take additional, optional arguments. Many of them work similarly on other kinds of values, to be explained later.

Lists (Exercises)

1. `['eggs', 'bread', 'tea']` ['eggs', 'bread', 'tea']
2. `nums = [4, 8, 15, 16, 23, 42]`

   `nums[3]` _____
3. `nums[2:4]` _____
4. `nums[1] = 99`

   `nums` _____
5. `len(nums)` _____
6. `sorted([4, 1, 3, 2])` _____
7. `nest = [[6, 7], [1, 1, 3, 8]]`

   `nest[1]` _____
8. `nest[1][2]` _____
9. `left = [1, 2, 3]`

   `right = [4, 5, 6]`

   `left + right` _____
10. `left * 2` _____
11. `3 in left` _____
12. `[2, 3] in left` _____

Lists

A *list* is a sequence of elements, delimited by square brackets `[` and `]`. The elements might be strings, numbers, or even other lists.

Lists behave much like strings. They can be accessed or sliced just like strings. Functions like `len` and `sorted` work the same way.

Unlike strings, lists can be modified. For example, if `ingredients` is a list, `ingredients[0] = 'olive oil'` replaces the first element of the list.

The `in` operator also behaves slightly differently. For a list `ls`, `a in ls` is true if `a` is an element of `ls`. For a string `s`, `a in s` is true if `a` is a substring of `s`.

Comparisons (Exercises)

1. `1 == 2` — `False`
2. `2 == 2` — _____
3. `2 < 3` — _____
4. `4 >= 3` — _____
5. `4 >= 4` — _____
6. `4 >= 5` — _____
7. `1 != 2` — _____
8. `'a' < 'c'` — _____
9. `'a' == 'A'` — _____
10. `'a' > 'A'` — _____
11. `'abc' < 'xyz'` — _____
12. `'rob' < 'robin'` — _____
13. `[1, 2, 3] < [1, 3, 5]` — _____
14. `[[1, 2], 3] < [[1, 3], 2]` — _____
15. `123 < 23` — _____
16. `'123' < '23'` — _____

Comparisons

There are several operators for comparing things:

- < less than
- <= less than or equal to
- == equal
- >= greater than or equal to
- > greater than
- `!=` not equal to

The resulting value is either `True` or `False`.

Note the difference between `a = 1`, which means "Make `a` a name for 1.", and `a == 1`, which means "Is `a` equal to 1?".

When comparing sequences like strings and lists, comparisons examine the first element of each. If they are equal, the comparison proceeds to the next element. If a difference is found, or one sequence runs out first, the sequences are not equal.

Characters are compared according to the Unicode specification. Digits are less than upper-case letters, which are in turn less than lower-case letters. Within these groups, smaller digits and earlier letters in the alphabet are considered less.

Types (Exercises)

1. `type(3)` <class 'int'>
2. `type(3.14159)` _____
3. `type(True)` _____
4. `type('hello')` _____
5. `type([1, 2, 3])` _____
6. `int(3.14159)` _____
7. `int('23')` _____
8. `float(3)` _____
9. `float('23')` _____
10. `str(3.14159)` _____
11. `str([1, 2, 3])` _____
12. `list('abc')` _____
13. `bool(0)` _____
14. `bool(1)` _____
15. `bool(0.0)` _____
16. `bool(-1.5)` _____
17. `bool('false')` _____
18. `bool('')` _____
19. `bool([1, 2, 3])` _____
20. `bool([])` _____

Types

Every value has a *type*. Conversely, each type is the set of values that have that type.

Types we have seen so far include:

- `int` Integer
- `float` Number including a decimal point or scientific notation, even if it happens to be an integer
- `str` String
- `list` List
- `bool` Boolean logical value: `True` or `False`

The built-in `type` function returns the type of its argument. For example, `type(20)` returns `<class 'int'>`. This is the `int` type; the reason for the more elaborate notation is beyond the scope of this book.

Each type is also the name of a function for converting values of other types. For example, `int('15')`, having been given the string `'15'`, returns the int `15`.

The type conversion functions generally behave as expected, but:

- Some conversions are not possible. `int('hello')` results in an error.
- Converting a float to an int discards anything after the decimal point.
- `bool` converts any zero or empty value to `False` and any other value to `True`.

Methods (Exercises)

1. `a = [3, 1, 4, 2]`

   `a.append(5)`

   `a` `[3, 1, 4, 2, 5]`

2. `a.extend([6, 7])`

   `a` _____

3. `a.insert(2, 8)`

   `a` _____

4. `a.index(4)` _____

5. `b = [1, 2, 3, 2, 4, 2, 5]`

   `b.count(2)` _____

6. `b.remove(2)`

   `b` _____

7. `b.sort()`

   `b` _____

8. `'ride a bike'.split()` _____

9. `'1,2,3'.split(',')` _____

10. `ls = ['eggs', 'bread', 'tea']`

    `', '.join(ls)` _____

11. `''.join(['a', 'b', 'c'])` _____

Methods

Methods are very similar to functions. Either might return a value or have some side effect like modifying the contents of a list. They are called slightly differently:

- `f(x, y)` is the function `f` being called and given the arguments `x` and `y`. This says to Python, "Hey function `f`, do your thing. You'll need `x` and `y`."
- `x.m(y)` is the method `m` being called *on* the object `x` and given the argument `y`. This says, "Hey object `x`, do `m`. You'll need `y`."

Methods are generally "about" the objects on which they are called.

List methods include:

- `append` modifies the list by adding its argument to the end.
- `extend` modifies the list by adding each element of its argument to the end.
- `insert` inserts an element at a specific index, moving everything after that one position to the right.
- `index` returns the position where the argument first appears in the list.
- `count` returns the number of times the argument appears in the list.
- `remove` removes the first appearance of the argument from the list.
- `sort` modifies the list by sorting it. This is different from the `sorted` function, which creates a new list.

String methods include:

- `split` returns a list of strings found by breaking the string apart. By default, it looks for spaces to break the string into words, but some other separator can be given as an argument.
- `join` creates a string from a list of strings. The string on which the method is called is used as the separator.

List Comprehensions (Exercises)

1. `a = [1, 4, 2, 3]`

   `[i * 2 for i in a]` `[2, 8, 4, 6]`

2. `[i + 1 for i in a]` _____

3. `[5 for i in a]` _____

4. `[i for i in a if i > 2]` _____

5. `s = 'python'`

   `vowels = 'aeiouy'`

   `[c for c in s if c in vowels]` _____

List Comprehensions

A *list comprehension* is a way of converting a list (or other sequence) into another list. The general form is:

`[`*`expr`* `for` *`var`* `in` *`sequence`*`]`

The value of this is found by assigning `var` to each element of `sequence` in turn, then evaluating `expr` (an expression which may involve `var`).

Optionally, a list comprehension can also include a condition:

`[`*`expr`* `for` *`var`* `in` *`sequence`* `if` *`condition`*`]`

Elements for which this condition is false are omitted from the result.

F-Strings (Exercises)

1.
```
n = 3
x = 2.71828
f'The number is {n}'
```
'The number is 3'

2. `f'2 + 2 = {2 + 2}'` _____

3. `f'{x}'` _____

4. `f'{x:.2}'` _____

5. `f'{2.1:.4}'` _____

6.
```
word = 'the'
f'{word:<5}'
```
_____

7. `f'{word:>5}'` _____

8. `f'{x:>6.3}'` _____

9.
```
nums = [2, 40, 1, 353]
s = [f'{n:<4}' for n in nums]
''.join(s)
```
_____

F-Strings

If a string is preceded by `f` (just outside the quotation marks), anything within curly braces `{}` inside that string is evaluated rather than taken as literal text.

A number of specifications can be provided, inside the curly braces after a colon `:`, for how the resulting value should be formatted. Specifically:

- '.' and a number specifies the maximum number of digits to show after the decimal point.
- '<' and a number adds spaces to pad the representation to the specified total length. The value is therefore left justified, which can be handy when printing tables.
- > and a number similarly adds spaces, but at the beginning, giving right justification.

Sets (Exercises)

1. `{1, 2, 3}` — {1, 2, 3}
2. `set([1, 2, 3])` —
3. `set([3, 2, 1])` —
4. `set([1, 2, 3, 1, 2, 3])` —
5. `set()` —
6. `{1, 2, 3} == {3, 2, 1}` —
7. `s  = {6, 2, 5}`

   `5 in s` —
8. `len(s)` —
9. `s.add(4)`

   `s` —
10. `s.remove(5)`

    `s` —
11. `a = {1, 2, 3}`

    `b = {2, 3, 4}`

    `a & b` —
12. `a | b` —

Sets

A *set* is a collection of items, delimited by curly braces `{` and `}`. It is very similar to a list, except that:

- A set cannot contain duplicate items.
- The order of the items in a set does not matter. Two sets are equal if they contain the exactly same elements.

Sets of very small integers tend to be shown in increasing order, but in general the display order is unpredictable and should not be relied on. This is a result of fancy data structures Python uses behind the scenes to make set operations efficient.

A set can be created explicitly using curly braces or using the `set` function. Calling `set` with no argument returns the empty set, displayed as `set()`.

As with strings and lists, the `in` operator can be used to check if something is in a set. `len` returns the size of a set.

The set methods `add` and `remove` modify the set on which they are called, adding or removing an item.

The `&` operator finds the *intersection* of two sets: the set containing only elements that are in both sets.

The `|` operator finds the *union* of two sets: the set containing all elements that are in at least one of the sets.

Dictionaries (Exercises)

1. `d = {1:'i', 2:'ii', 3:'iii'}`

   `d` {1: 'i', 2: 'ii', 3: 'iii'}
2. `d[2]` _____
3. `d[3] = 'many'`

   `d[3]` _____
4. `d[2] = 'many'`

   `d` _____
5. `3 in d` _____
6. `len(d)` _____
7. `del d[2]`

   `d` _____
8. `{}` _____
9. `type({})` _____
10. `d = {1:'a', 2:'b', 3:'c'}`

    `[d[n] for n in d]` _____

Dictionaries

A *dictionary* (of type `dict`) stores a set of *keys* and associated *values*. The analogy is to a dictionary for a natural human language, in which the keys are words and the values are their definitions.

Each key and its value are separated by a colon. The entire dictionary is delimited by curly braces `{` and `}`. As with sets, the order of keys in a dictionary doesn't matter and shouldn't be depended on.

Pairs can be looked up or replaced using square brackets, as with a list. With a dict, the square brackets contain the key in question rather than an index.

The `in` operator indicates if a given key is present. `len` returns the size of the dict.

To remove a pair from a dict, use the `del` operator:

`del` *`dictonary`*`[`*`key`*`]`

`[]` is obviously an empty list. `{}` is an empty dict. The empty set is represented as `set()`.

The keys of a dictionary can be iterated through using a list comprehension.

Equality (Exercises)

1. `a = [1, 2, 3]`

   `b = a`

   `c = [1, 2, 3]`

   `d = a.copy()`

   `a == b` True

2. `a == c` _____

3. `a == d` _____

4. `a[0] = 0`

   `a` _____

5. `b` _____

6. `c` _____

7. `d` _____

8. `a == b` _____

9. `a == c` _____

10. `a == d` _____

11. `a = [1, 2, 3]`

    `b = a`

    `a = [4, 5, 6]`

    `b` _____

Equality

What does it mean for two variables `x` and `y` to be equal? There are two possibilities:

1. `x` and `y` are two names for *the same object*
2. `x` and `y` are names for objects that *have the same content*, whether or not they are the same object

Python's == operator uses the second interpretation.

Usually the difference doesn't matter, but for objects than can be modified (like lists, sets, and dictionaries), it does. Specifically, suppose `x` and `y` are *aliases* for the same object. Modifying the object named by `x` also modifies the object named by 'y', because they are the same object.

Assigning `x` to name *a different object* does not modify `y` in this situation.

The `copy` method, provided by lists, sets, and dicts, makes a copy of an object.

Tuples (Exercises)

1. `t = (1, 2, 3)`

   `t` (1, 2, 3)
2. `()` _____
3. `type(())` _____
4. `x, y, z = t`

   `x` _____
5. `y` _____
6. `u = x, y`

   `u` _____
7. `a = 1`

   `b = 2`

   `a, b = b, a`

   `a` _____
8. `b` _____
9. `(2 + 2 + 2)` _____
10. `(2 + 2)` _____
11. `(2)` _____
12. `(2, 2, 2)` _____
13. `(2, 2)` _____
14. `(2,)` _____

Tuples

A *tuple* is almost exactly the same as a list, but it is delimited by round parentheses `(` and `)`. The difference is that a tuple, unlike a list, is *immutable*: it cannot be modified. This is helpful for a couple of reasons:

- The same tuple can have several names without any risk of the aliasing problems described previously under Equality.
- Because of the underlying efficient data structures, set elements and dictionary keys have to be immutable. Tuples work for this but lists don't.

Several variables can be assigned to the elements of a tuple with one assignment statement. The tuple is said to be *unpacked*. Conversely, if several values (separated by commas) appear on the right side of an assignment statement, they are *packed* into a tuple.

A tuple with exactly one element has a comma after that element. Otherwise, something like `(2)` would be ambiguous.

Defining Functions (Exercises)

1. ```
   def add(x, y):
       return x + y
   add(2, 3)
   ```
   5

2. ```
   def hypotenuse(a, b):
       a2 = a ** 2
       b2 = b ** 2
       return (a2 + b2) ** 0.5
   hypotenuse(3, 4)
   ```
   _____

3. ```
   x = 3
   def f():
       x = 4
   f()
   x
   ```
   _____

4. ```
   def g():
       y = x
       return y
   g()
   ```
   _____

5. ```
   def replace_first(ls):
       ls[0] = 100
   nums = [1, 2, 3]
   replace_first(nums)
   nums
   ```
   _____
   ```

Defining Functions

In addition to the many functions built into Python, you can define your own. The general format is

```
def name(parameter(s)):
    statement(s)
```

where zero or more parameters are separated by commas and one or more statements are on successive lines.

Carefully note the colon `:` at the end of the first line and the indentation of the statements within the definition.

When the function is called, the parameters are assigned as names for whatever values are passed in as arguments. The statements are then executed in order.

If the function reaches a `return` statement, the value specified is returned as the value of the function call.

The parameters of a function are only visible inside that function. There are several subtleties to this:

- Even if a parameter happens to have the same name as a global variable, setting it inside the function does not affect the global variable. It is said to *shadow* the global variable, which is no longer visible inside the function.
- Assigning a value inside a function creates a new *local variable*, which shadows any global variable with the same name.
- Unshadowed global variables can be *read* inside a function.
- If a parameter or local variable names a mutable object (like a list), modifying that object does affect other names for it outside the function.

If Statements (Exercises)

1.
```
x = 1
if 1 < 2:
    x = 2
x
```
2

2.
```
if 4 in {1, 2, 3}:
    a = 1
else:
    a = 2
a
```
_____

3.
```
def accessory(temp, rain):
    if temp >= 70:
        if rain:
            return 'umbrella'
        else:
            return 'beverage'
    elif temp >= 40:
        return 'jacket'
    else:
        return 'coat'
accessory(59, True)
```
_____

4. `accessory(70, False)` _____

5. `accessory(33, False)` _____

6. `accessory(72, True)` _____

If Statements

An `if` statement executes one or more statements if and only if some condition is true. The general form is:

```
if condition:
    statement(s)
```

An optional `else` clause provides alternate statements to be executed if the condition is *not* true:

```
if condition:
    statement(s)
else:
    statement(s)
```

Even more conditions can be provided by including `elif` (short for "else if") clauses:

```
if condition1:
    statement(s)
elif condition2:
    statement(s)
else:
    statement(s)
```

The statement stops after the first true condition.

While Loops (Exercises)

1.
```
x = 1
sum = 0
while x <= 5:
    sum = sum + x
    x = x + 1
sum
```
15

2.
```
word = 'indivisible'
i = 0
while i < 5:
    i = i + 1
word[:i]
```
_____

3.
```
result = ''
i = 0
while i < 5:
    result = result + f'<{i}'
    i = i + 1
    result = result + '>'
result
```
_____

4.
```
nums = [1, 2, 3]
[n * 2 for n in nums]
```
_____

5.
```
result = []
i = 0
while i < len(nums):
    result.append(nums[i] * 2)
    i = i + 1
result
```
_____

While Loops

A `while` loop allows one or more statements to be executed repeatedly. The general form is:

```
while condition:
    statement(s)
```

The condition is checked at the beginning of each pass through the loop. If it is true, the statements are executed. The condition is then checked again, and so on.

The condition is only checked at the beginning of each pass through the loop. The loop does not end mid-pass merely because the condition has become false.

A `while` loop can replicate what a list comprehension does, but the list comprehension is more convenient for this specific task.

For Loops (Exercises)

1.
```
sum = 0
for i in [1, 2, 3, 4, 5]:
    sum = sum + i
sum
```
15

2.
```
total = 0
for c in 'indivisible':
    if c == 'i':
        total = total + 1
total
```
_____

3.
```
nums = [1, 2, 3]
[n * 2 for n in nums]
```
_____

4.
```
result = []
for n in nums:
    result.append(n * 2)
result
```
_____

For Loops

A `for` loop is in between a list comprehension and a `while` loop in terms of generality and convenience.

The general form is:

```
for var in sequence:
    statement(s)
```

Like a list comprehension, it assigns `var` to each element of `sequence` in turn, then executes the statements.

String Indexing (Solutions)

1. `'python'[0]` `'p'`
2. `'python'[1]` `'y'`
3. `'python'[3]` `'h'`
4. `'python'[-1]` `'n'`
5. `'python'[-3]` `'h'`

String Slicing (Solutions)

1. `'abandon'[0:2]` `'ab'`
2. `'abandon'[1:5]` `'band'`
3. `'infinite'[2:100]` `'finite'`
4. `'incandescent'[3:8:2]` `'ads'`
5. `'airspace'[:3]` `'air'`
6. `'airspace'[3:]` `'space'`
7. `'remote'[4:0:-1]` `'tome'`
8. `'duplicate'[::]` `'duplicate'`
9. `'straw'[::-1]` `'warts'`

Arithmetic Operators (Solutions)

1. `2 + 2` 4
2. `2 + 3` 5
3. `4 - 1` 3
4. `3 - 8` -5
5. `2 * 4` 8
6. `8 / 2` 4.0
7. `7 / 2` 3.5
8. `7 // 2` 3
9. `34 // 10` 3
10. `34 % 10` 4
11. `2 ** 3` 8
12. `10 ** 20` 100000000000000000000
13. `2 + 3 * 5` 17
14. `2 + (3 * 5)` 17
15. `(2 + 3) * 5` 25
16. `1 + 2 * 3 - 4 ** 2 / 4` 3.0

Variables (Solutions)

1. `x = 3`

   `x` 3

2. `x + 2` 5

3. `y = 4`

   `y` 4

4. `x + y` 7

5. `color = 'blue'`

   `color` `'blue'`

6. `score = 5`

   `score = 10`

   `score` 10

7. `a = 1`

   `b = a`

   `a = 2`

   `b` 1

8. `n = 10`

   `n = n + 1`

   `n` 11

String Operators (Solutions)

1. `'hot' + 'dog'` `'hotdog'`
2. `'shoe' + 'string'` `'shoestring'`
3. `'ma' * 2` `'mama'`
4. `'co' * 2 + 'a'` `'cocoa'`
5. `'i' in 'team'` `False`
6. `'on' in ('no' * 2)` `True`
7.
```
a = 'brown'
b = 'toad'
a[1:3] + b[2:]
```
`'road'`

Quotation Marks (Solutions)

1.	`"hello"`	`'hello'`
2.	`'goodbye'`	`'goodbye'`
3.	`"don't"`	`"don't"`
4.	`'Eddie "The Eagle" Edwards'`	`'Eddie "The Eagle" Edwards'`
5.	`'You "can\'t" avoid it.'`	`'You "can\'t" avoid it.'`
6.	`"... or \"can\" you?"`	`'... or "can" you?'`
7.	`'Maybe \'sometimes\' you can.'`	`"Maybe 'sometimes' you can."`

Calling Functions (Solutions)

1. `abs(-3)` 3
2. `abs(12)` 12
3. `len('python')` 6
4. `max(2, 3)` 3
5. `min(2, 3)` 2
6. `round(3.14159)` 3
7.
```
a = 'snake'
b = 'language'
max(len(a), len(b))
```
8
8.
```
s = 'python'
s[0:len(s)]
```
'python'
9. `max(1, min(2, 3))` 2
10. `sorted(s)` ['h', 'n', 'o', 'p', 't', 'y']
11. `s` 'python'

Lists (Solutions)

1. `['eggs', 'bread', 'tea']` — `['eggs', 'bread', 'tea']`
2. `nums = [4, 8, 15, 16, 23, 42]`

   `nums[3]` — `16`
3. `nums[2:4]` — `[15, 16]`
4. `nums[1] = 99`

   `nums` — `[4, 99, 15, 16, 23, 42]`
5. `len(nums)` — `6`
6. `sorted([4, 1, 3, 2])` — `[1, 2, 3, 4]`
7. `nest = [[6, 7], [1, 1, 3, 8]]`

   `nest[1]` — `[1, 1, 3, 8]`
8. `nest[1][2]` — `3`
9. `left = [1, 2, 3]`

   `right = [4, 5, 6]`

   `left + right` — `[1, 2, 3, 4, 5, 6]`
10. `left * 2` — `[1, 2, 3, 1, 2, 3]`
11. `3 in left` — `True`
12. `[2, 3] in left` — `False`

Comparisons (Solutions)

1. `1 == 2` — False
2. `2 == 2` — True
3. `2 < 3` — True
4. `4 >= 3` — True
5. `4 >= 4` — True
6. `4 >= 5` — False
7. `1 != 2` — True
8. `'a' < 'c'` — True
9. `'a' == 'A'` — False
10. `'a' > 'A'` — True
11. `'abc' < 'xyz'` — True
12. `'rob' < 'robin'` — True
13. `[1, 2, 3] < [1, 3, 5]` — True
14. `[[1, 2], 3] < [[1, 3], 2]` — True
15. `123 < 23` — False
16. `'123' < '23'` — True

Types (Solutions)

1. `type(3)` <class 'int'>
2. `type(3.14159)` <class 'float'>
3. `type(True)` <class 'bool'>
4. `type('hello')` <class 'str'>
5. `type([1, 2, 3])` <class 'list'>
6. `int(3.14159)` 3
7. `int('23')` 23
8. `float(3)` 3.0
9. `float('23')` 23.0
10. `str(3.14159)` '3.14159'
11. `str([1, 2, 3])` '[1, 2, 3]'
12. `list('abc')` ['a', 'b', 'c']
13. `bool(0)` False
14. `bool(1)` True
15. `bool(0.0)` False
16. `bool(-1.5)` True
17. `bool('false')` True
18. `bool('')` False
19. `bool([1, 2, 3])` True
20. `bool([])` False

Methods (Solutions)

1. `a = [3, 1, 4, 2]`

   `a.append(5)`

   `a` `[3, 1, 4, 2, 5]`
2. `a.extend([6, 7])`

   `a` `[3, 1, 4, 2, 5, 6, 7]`
3. `a.insert(2, 8)`

   `a` `[3, 1, 8, 4, 2, 5, 6, 7]`
4. `a.index(4)` `3`
5. `b = [1, 2, 3, 2, 4, 2, 5]`

   `b.count(2)` `3`
6. `b.remove(2)`

   `b` `[1, 3, 2, 4, 2, 5]`
7. `b.sort()`

   `b` `[1, 2, 2, 3, 4, 5]`
8. `'ride a bike'.split()` `['ride', 'a', 'bike']`
9. `'1,2,3'.split(',')` `['1', '2', '3']`
10. `ls = ['eggs', 'bread', 'tea']`

    `', '.join(ls)` `'eggs, bread, tea'`
11. `''.join(['a', 'b', 'c'])` `'abc'`

List Comprehensions (Solutions)

1. `a = [1, 4, 2, 3]`

   `[i * 2 for i in a]` `[2, 8, 4, 6]`
2. `[i + 1 for i in a]` `[2, 5, 3, 4]`
3. `[5 for i in a]` `[5, 5, 5, 5]`
4. `[i for i in a if i > 2]` `[4, 3]`
5. `s = 'python'`

   `vowels = 'aeiouy'`

   `[c for c in s if c in vowels]` `['y', 'o']`

F-Strings (Solutions)

1. ```
   n = 3
   x = 2.71828
   f'The number is {n}'
   ```
   `'The number is 3'`
2. ```
   f'2 + 2 = {2 + 2}'
   ```
   `'2 + 2 = 4'`
3. ```
   f'{x}'
   ```
   `'2.71828'`
4. ```
   f'{x:.2}'
   ```
   `'2.7'`
5. ```
   f'{2.1:.4}'
   ```
   `'2.1'`
6. ```
   word = 'the'
   f'{word:<5}'
   ```
   `'the  '`
7. ```
   f'{word:>5}'
   ```
   `'  the'`
8. ```
   f'{x:>6.3}'
   ```
   `'  2.72'`
9. ```
   nums = [2, 40, 1, 353]
   s = [f'{n:<4}' for n in nums]
   ''.join(s)
   ```
   `'2   40  1   353 '`
   ```

Sets (Solutions)

1. `{1, 2, 3}` — {1, 2, 3}
2. `set([1, 2, 3])` — {1, 2, 3}
3. `set([3, 2, 1])` — {1, 2, 3}
4. `set([1, 2, 3, 1, 2, 3])` — {1, 2, 3}
5. `set()` — set()
6. `{1, 2, 3} == {3, 2, 1}` — True
7. `s  = {6, 2, 5}`

   `5 in s` — True
8. `len(s)` — 3
9. `s.add(4)`

   `s` — {2, 4, 5, 6}
10. `s.remove(5)`

    `s` — {2, 4, 6}
11. `a = {1, 2, 3}`

    `b = {2, 3, 4}`

    `a & b` — {2, 3}
12. `a | b` — {1, 2, 3, 4}

Dictionaries (Solutions)

1. `d = {1:'i', 2:'ii', 3:'iii'}`

   `d` → `{1: 'i', 2: 'ii', 3: 'iii'}`
2. `d[2]` → `'ii'`
3. `d[3] = 'many'`

   `d[3]` → `'many'`
4. `d[2] = 'many'`

   `d` → `{1: 'i', 2: 'many', 3: 'many'}`
5. `3 in d` → `True`
6. `len(d)` → `3`
7. `del d[2]`

   `d` → `{1: 'i', 3: 'many'}`
8. `{}` → `{}`
9. `type({})` → `<class 'dict'>`
10. `d = {1:'a', 2:'b', 3:'c'}`

    `[d[n] for n in d]` → `['a', 'b', 'c']`

Equality (Solutions)

1. `a = [1, 2, 3]`

   `b = a`

   `c = [1, 2, 3]`

   `d = a.copy()`

   `a == b` — True
2. `a == c` — True
3. `a == d` — True
4. `a[0] = 0`

   `a` — [0, 2, 3]
5. `b` — [0, 2, 3]
6. `c` — [1, 2, 3]
7. `d` — [1, 2, 3]
8. `a == b` — True
9. `a == c` — False
10. `a == d` — False
11. `a = [1, 2, 3]`

    `b = a`

    `a = [4, 5, 6]`

    `b` — [1, 2, 3]

Tuples (Solutions)

1. `t = (1, 2, 3)`

   `t` — `(1, 2, 3)`
2. `()` — `()`
3. `type(())` — `<class 'tuple'>`
4. `x, y, z = t`

   `x` — `1`
5. `y` — `2`
6. `u = x, y`

   `u` — `(1, 2)`
7. `a = 1`

   `b = 2`

   `a, b = b, a`

   `a` — `2`
8. `b` — `1`
9. `(2 + 2 + 2)` — `6`
10. `(2 + 2)` — `4`
11. `(2)` — `2`
12. `(2, 2, 2)` — `(2, 2, 2)`
13. `(2, 2)` — `(2, 2)`
14. `(2,)` — `(2,)`

Defining Functions (Solutions)

1.
```
def add(x, y):
    return x + y
add(2, 3)
```
5

2.
```
def hypotenuse(a, b):
    a2 = a ** 2
    b2 = b ** 2
    return (a2 + b2) ** 0.5
hypotenuse(3, 4)
```
5.0

3.
```
x = 3
def f():
    x = 4
f()
x
```
3

4.
```
def g():
    y = x
    return y
g()
```
3

5.
```
def replace_first(ls):
    ls[0] = 100
nums = [1, 2, 3]
replace_first(nums)
nums
```
[100, 2, 3]

If Statements (Solutions)

1.
```
x = 1
if 1 < 2:
    x = 2
x
```
2

2.
```
if 4 in {1, 2, 3}:
    a = 1
else:
    a = 2
a
```
2

3.
```
def accessory(temp, rain):
    if temp >= 70:
        if rain:
            return 'umbrella'
        else:
            return 'beverage'
    elif temp >= 40:
        return 'jacket'
    else:
        return 'coat'
accessory(59, True)
```
'jacket'

4. `accessory(70, False)` 'beverage'

5. `accessory(33, False)` 'coat'

6. `accessory(72, True)` 'umbrella'

While Loops (Solutions)

1.
```
x = 1
sum = 0
while x <= 5:
    sum = sum + x
    x = x + 1
sum
```
15

2.
```
word = 'indivisible'
i = 0
while i < 5:
    i = i + 1
word[:i]
```
'indiv'

3.
```
result = ''
i = 0
while i < 5:
    result = result + f'<{i}'
    i = i + 1
    result = result + '>'
result
```
'<0><1><2><3><4>'

4.
```
nums = [1, 2, 3]
[n * 2 for n in nums]
```
[2, 4, 6]

5.
```
result = []
i = 0
while i < len(nums):
    result.append(nums[i] * 2)
    i = i + 1
result
```
[2, 4, 6]

For Loops (Solutions)

1.
```
sum = 0
for i in [1, 2, 3, 4, 5]:
    sum = sum + i
sum
```
15

2.
```
total = 0
for c in 'indivisible':
    if c == 'i':
        total = total + 1
total
```
4

3.
```
nums = [1, 2, 3]
[n * 2 for n in nums]
```
[2, 4, 6]

4.
```
result = []
for n in nums:
    result.append(n * 2)
result
```
[2, 4, 6]